

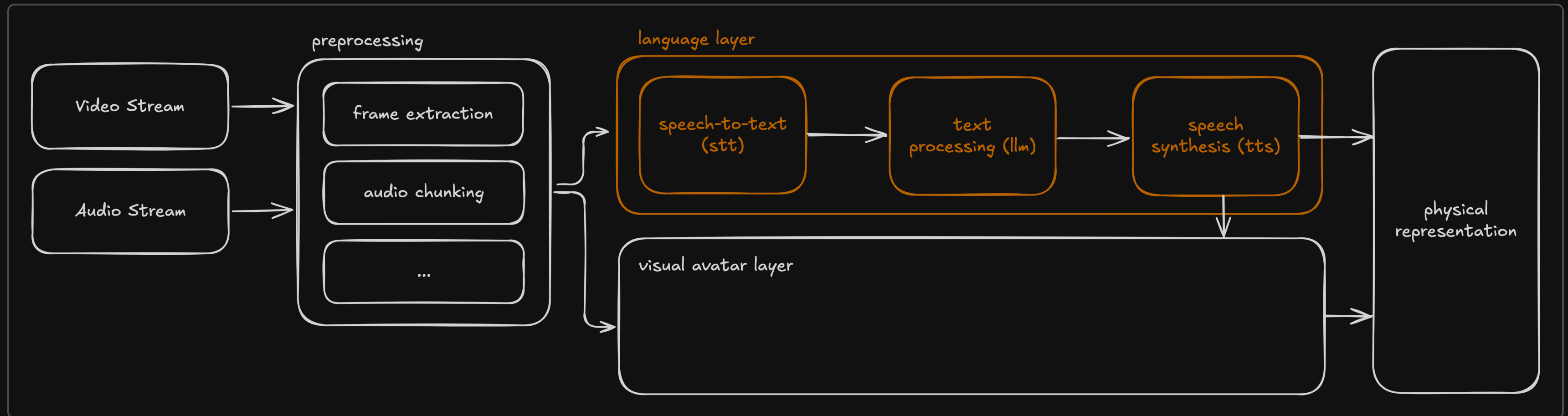
Kaspar 2028

Supervised by Prof. Dr. Lena Gieseke

K.ai: Entwurf Architektur

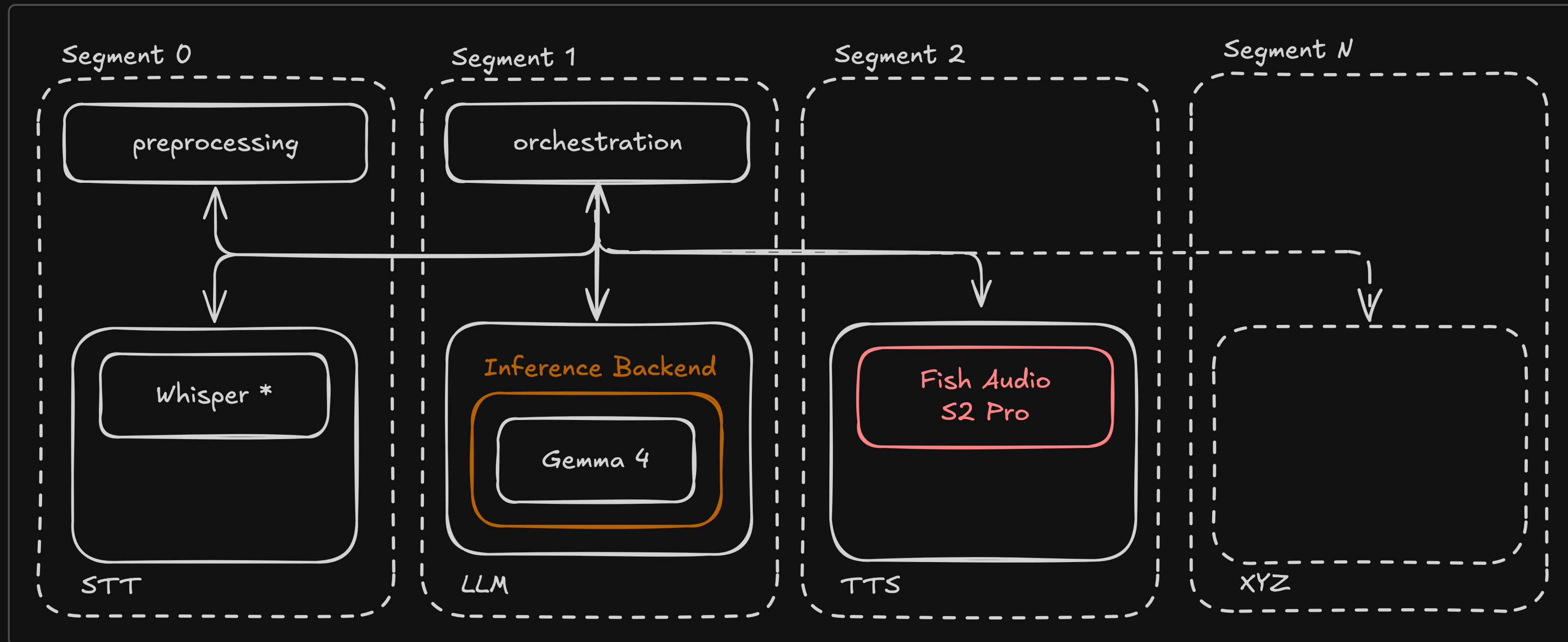
Initialer Umriss

Kernfunktionalität



- Latenz minimieren (Echtzeit-Pipeline)
- Multimodale Signalverarbeitung (Sehen & Hören)

Modularisierung



- Kürzeste Latenz
 - 1 spezialisierte Workstation mit mehreren dedizierten GPUs für versch. Module
 - Datenaustausch durch CUDA IPC (GPU-to-GPU) oder Shared Memory
 - Höchste Kosten & höchste Komplexität (nicht umsetzbar)
- Größte Flexibilität
 - Jedes Segment als Server mit `openai api` kompatiblen Endpunkt
 - IPC über lokales Netzwerk (zB.: Websockets)
 - gutes Mapping auf vorhandene Hardware (Verteilung auf mehrere Workstations)
 - nahtloses Fallback auf gehostete APIs (zB.: OpenAI Realtime API)

Orchestrierung

LiveKit Agents

- spezifisch entwickelt für STT → LMM → TTS Pipelines
- nahtlose Kompatibilität mit `openai api`

STT Modul

Whisper

- faster-whisper
- whisper-flow
- Möglicherweise obsolet wenn Gemma 4 Audio Input genutzt werden kann

LLM Modul

Gemma 4

- Multimodaler Input direkt in das LLM
- Spezifische Optimierungen für lokale Inferenz
 - TurboQuant
 - Per-Layer Embeddings
- Apache 2.0 Lizenz
- Verfügbar in `vLLM` und `ollama (llama.cpp)`

Linux ?

- Deutlich besserer Support für Inferenz-Provider (`SGLang` & `vLLM`)
- Mangelnde Erfahrung mit WSL oder Docker bzw. Linux allgemein (Philip)

TTS Modul

Fish Audio S2

- Unklare Lizenzierung (Nur zu Forschungszwecken) ! <https://github.com/fishaudio/fish-speech/blob/main/LICENSE>